

Datové struktury a algoritmy

Přednáška 1, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Motivace pro studium DSA

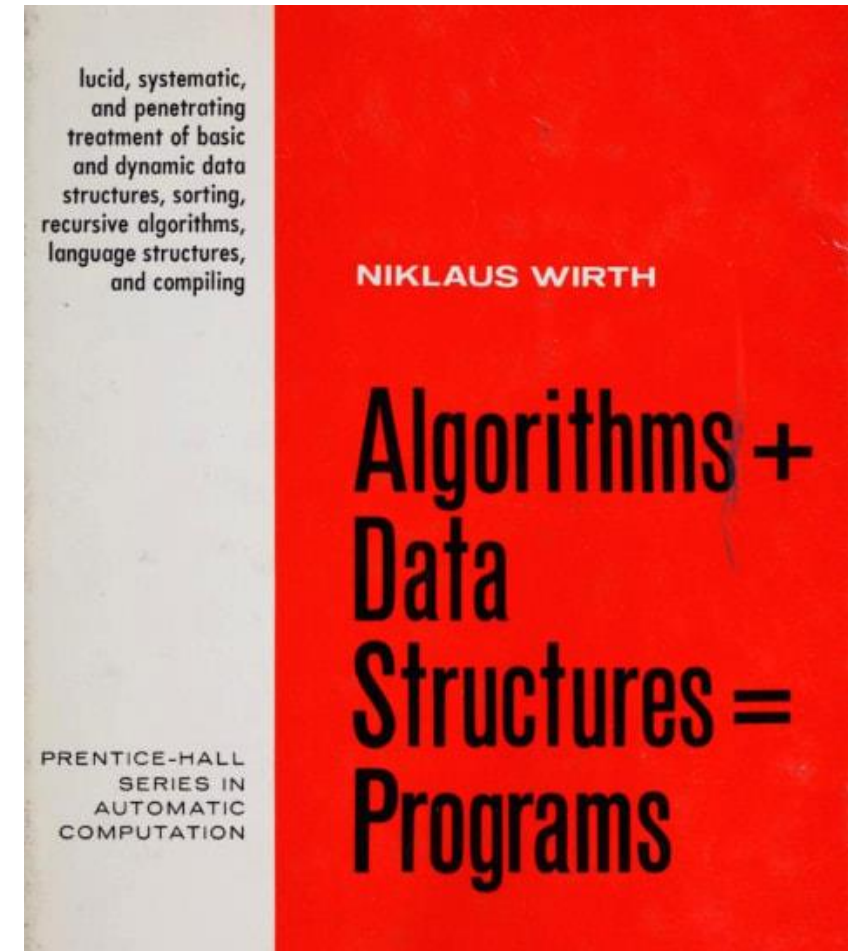
N. Wirth:

Algoritmy + datové struktury = programy

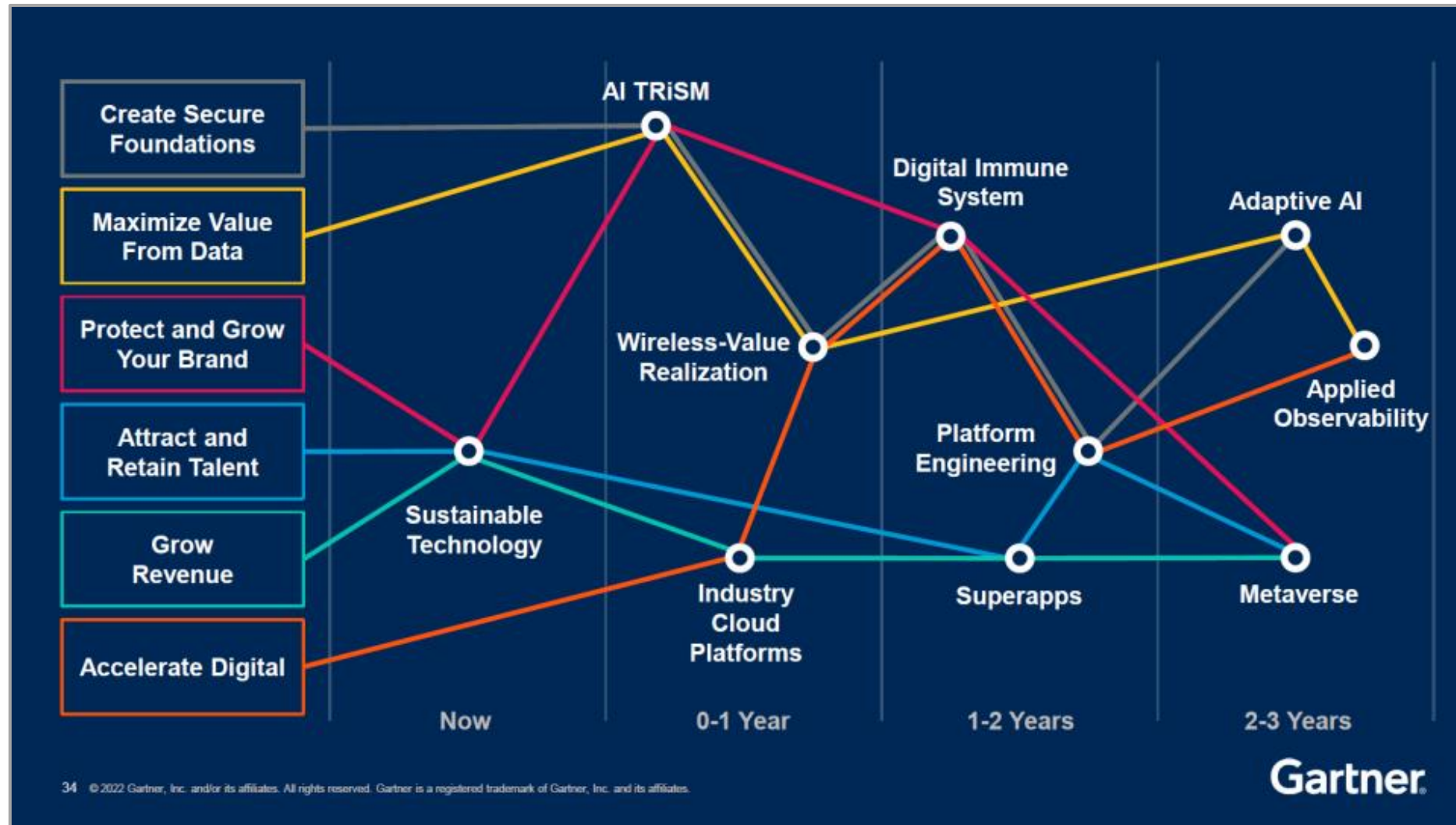
Většina programů spotřebuje daleko více času na **manipulaci s daty** než na výpočty a komunikaci.

- Vytvoření, uložení, vyhledání, editace, smazání

V tomto kurzu půjde o opakování, systemizaci a rozšíření stávajících teoretických znalostí a **programátorských dovedností**, týkajících se způsobu reprezentace dat a jejich zpracování s cílem **minimalizovat časovou a paměťovou náročnost** navržených postupů.

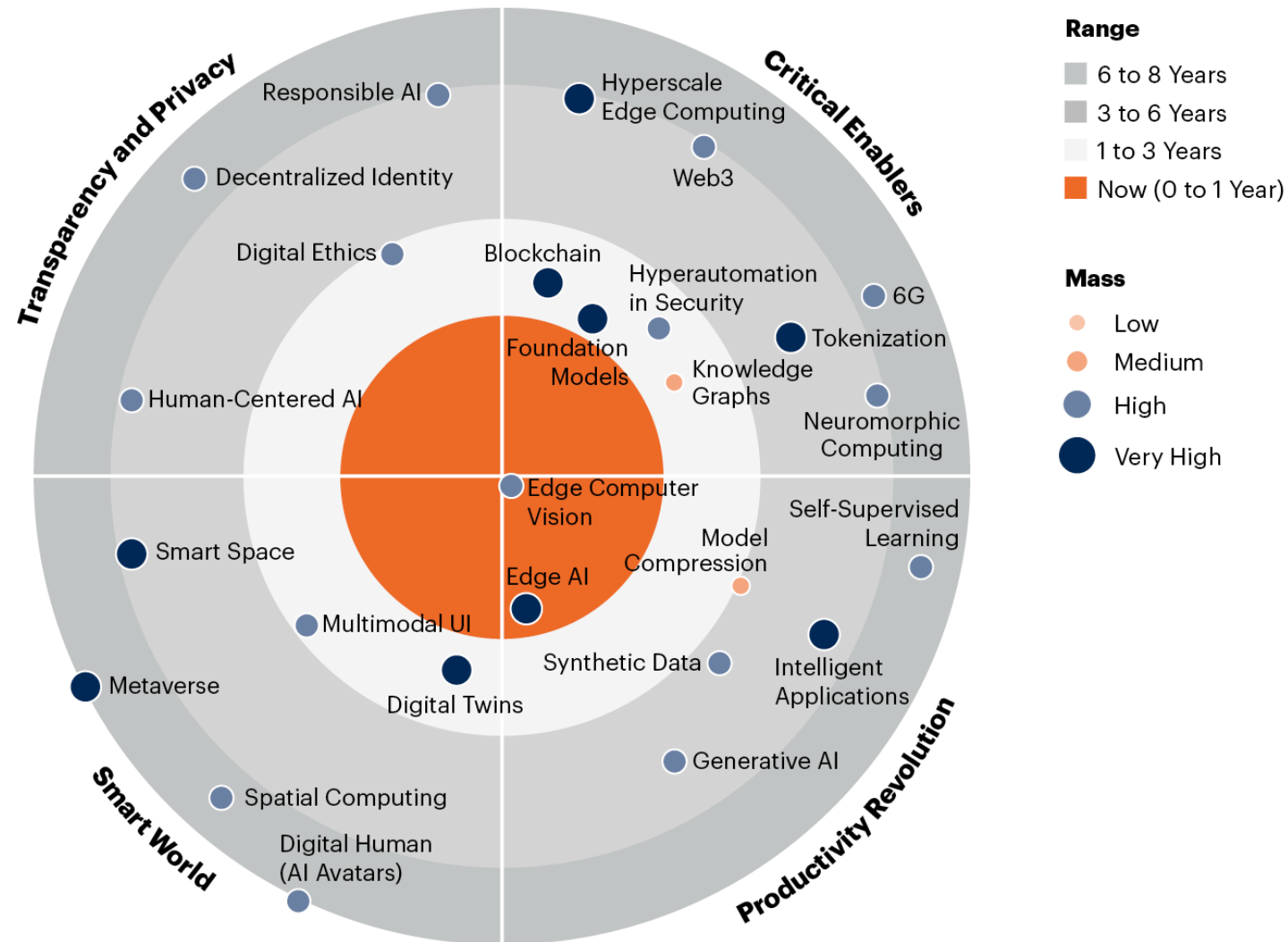


Gartner: strategické ICT trendy od 2023, Syntéza řešení



Zdroje: <https://www.pcmag.com/news/gartners-top-strategic-technology-trends-for-2023>

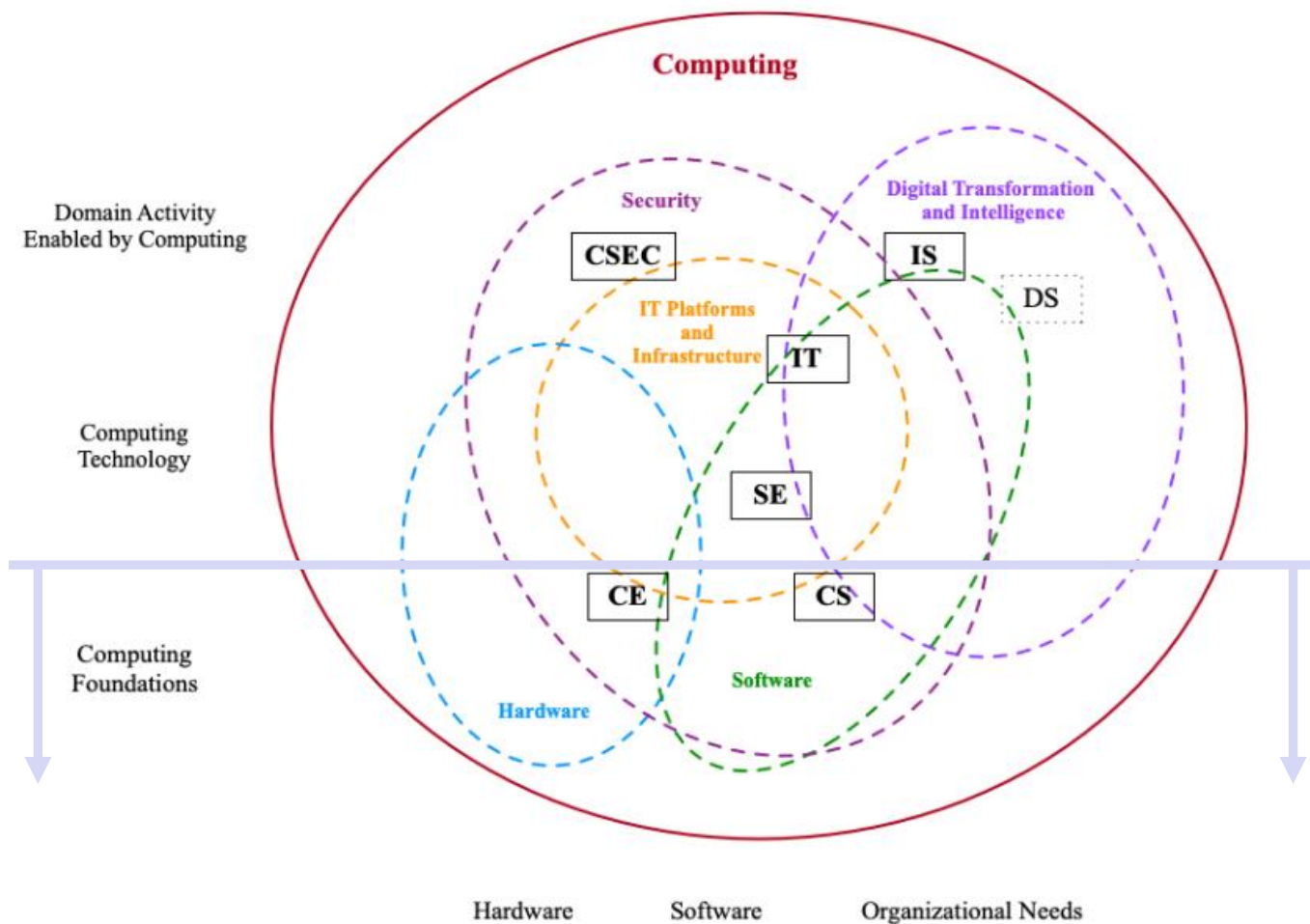
Gartner: strategické ICT trendy od 2023, Radar technológií



Zdroje: <https://www.gartner.com/en/articles/4-emerging-technologies-you-need-to-know-about>

Aktuální spektrum informatického vzdělávání

(2020)



CE: Computer Engineering

CS: Computer Science

CSEC: Cybersecurity

IS: Information Systems

IT: Information Technology

SE: Software Engineering

DS: Data science

Kurz DSA ve studijním programu AI

- DSA je navazujícím programátorským předmětem

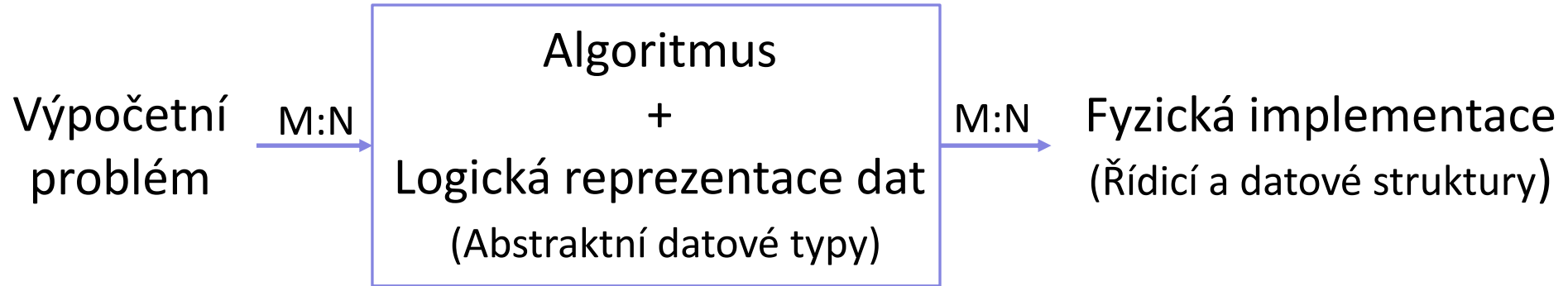
ZSP → OOP → **DSA** → PJP

Programování se nedá naučit pasivně, aktivní přístup studentů je nezbytný!

- Rozdělení získaných znalostí a dovedností
 - **Přednášky** osvětlí základní teoretické principy a souvislosti. Jejich součástí budou:
 - Povinné individuální prezentace
 - Řešení krátkých příkladů nebo problémů za **bonusové body**, dostupné následně v IS
 - Prezentace vlastních praktických poznatků je vítána
 - Praktické dovednosti si osvojíte formou **cvičení** a **semestrálního projektu**

Souhrnné informace o obsahu kurzu

- Obecná struktura řešení výpočetních problémů:



- Programovací jazyky

- **Imperativní** (JAK): procedurální, **objektové**, paralelní, technologické, ...
- Deklarativní (CO): databázové, funkcionální, logické, ...

Při jejich použití se vždy snažíme o celkovou optimálnost řešení.

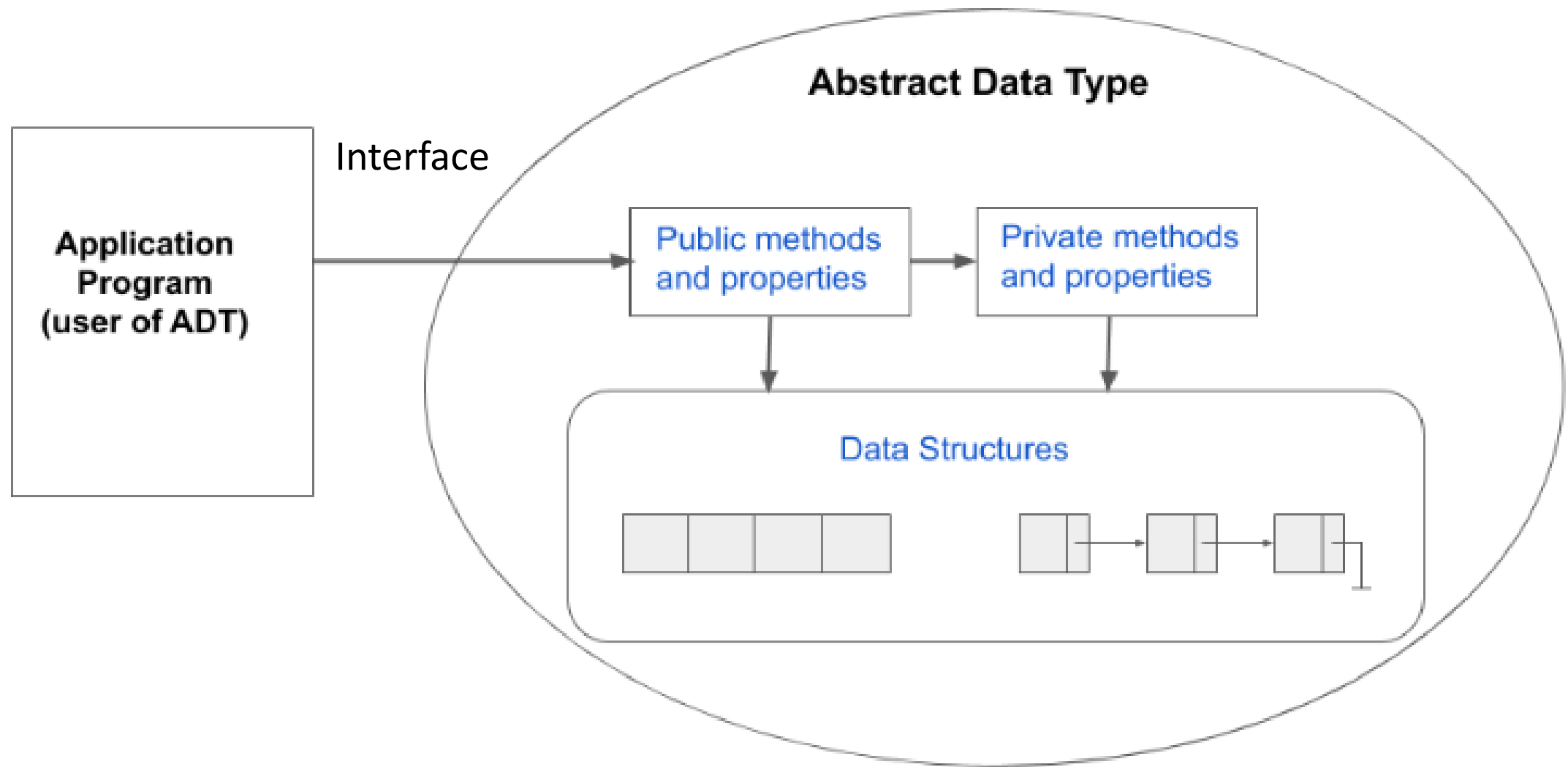
- Témata v DSA

1. Algoritmy a abstraktní datové typy
2. Výpočetní složitost
3. Algoritmy řazení
4. Algoritmy vyhledávání

Abstraktní a konkrétní datové typy a struktury podrobněji

- Algoritmus i **abstraktní datový typ** jsou **LOGICKÉ** kategorie (CO)
- Základní/uživatelské **datové typy/struktury/kontejnery** jsou **FYZICKÉ** kategorie (JAK)
- Konkrétní problémy lze řešit zcela samostatně nebo jejich asociací s existujícími zdroji (algoritmy, datovými typy či strukturami). Ve druhém případě se obvykle toleruje rozumná míra modifikace původního zadání.
- Zásadní myšlenkou této standardizace je vzájemná kombinovatelnost libovolných algoritmů a datových struktur, což vede na jejich unifikované a tím pádem i intuitivní použití. Toto bývá obvykle zaručeno na úrovni standardních knihoven jednotlivých programovacích jazyků.
- Kurz DSA ukazuje různé způsoby **VLASTNÍ implementace** ADT pomocí kombinace vhodných datových struktur a algoritmů a hodnocení jejich účinnosti (čas, paměť). Na závěr kurzu je užití ADT demonstrováno na knihovně STL jazyka C++.

Vztah ADT a DS



Příklad úplného návrhu a realizace ADT List

1. Signatura/reprezentace

<code>init: -> List</code>	prázdný seznam
<code>insert(_, _): List, T -> List</code>	vlož před kurzor
<code>remove(_): List -> List</code>	odeber
<code>removePrev(_): List -> List</code>	odeber před kurzorem
<code>toNext(_): List -> List</code>	kurzor na další
<code>toPrev(_): List -> List</code>	kurzor na předchozí
<code>toBegin(_): List -> List</code>	kurzor na začátek
<code>toEnd(_): List -> List</code>	kurzor na konec
<code>empty(_): List -> bool</code>	je seznam prázdný?
<code>atBegin(_): List -> bool</code>	je kurzor na začátku?
<code>atEnd(_): List -> bool</code>	je kurzor na konci?
<code>read(_): List -> T</code>	hodnota na kurzoru

2. Obecná implementační struktura

- Např. obousměrný spojový seznam



3. Konkrétní způsob SW realizace

- Vlastní třída *List* s konkrétní soukromou a veřejnou částí včetně definice jednotlivých metod

```
class List {
public:
    List();
    ~List();
    void insert(T); // vlož před kurzor
    void remove(); // odstran prvek na kurzoru
    void removePrev(); // odstran prvek před kurzorem
    void toNext(); // kurzor na další prvek
    void toPrev(); // kurzor na předchozí prvek
    void toBegin(); // kurzor na začátek
    void toEnd(); // kurzor na konec
    bool empty(); // je seznam prázdný?
    bool atBegin(); // je kurzor na začátku?
    bool atEnd(); // je kurzor na konci?
    bool read(T&); // čtení z pozice kurzoru
    friend ostream& operator<<(ostream&, const List&);
};
```

```
private:
    struct Node {
        T val;
        Node *next, *prev;
        Node(T x, Node *n) { // n nesmí být NULL
            val = x; next = n; prev = NULL;
        }
        Node() {next = NULL; prev = NULL;}
    };
    Node *begin;
    Node *tail;
    Node *mark;
    List(const List &);
    List& operator=(const List&);
};
```

Základní administrativa: zápočet, povinné aktivity, zkouška

- Zápočet: docházka + odevzdání (CV + SP)
 - Uděluje cvičící
- Povinné aktivity: 2x prezentace (IT + SP) + test
 - Registruje přednášející
- Zkoušku lze vykonat i bez zápočtu, avšak bez něj nemůže být uznána
 - Hodnotí přednášející
 - Bude i možnost cvičné on-line zkoušky
- **Další podrobnosti a termíny v Průvodci studiem (Moodle)**

Dnešní a související navazující témata

- **Algoritmy** a způsoby jejich reprezentace
- Abstraktní **datové typy** a způsoby jejich reprezentace
- Programovací jazyky, generující spustitelné **programy**

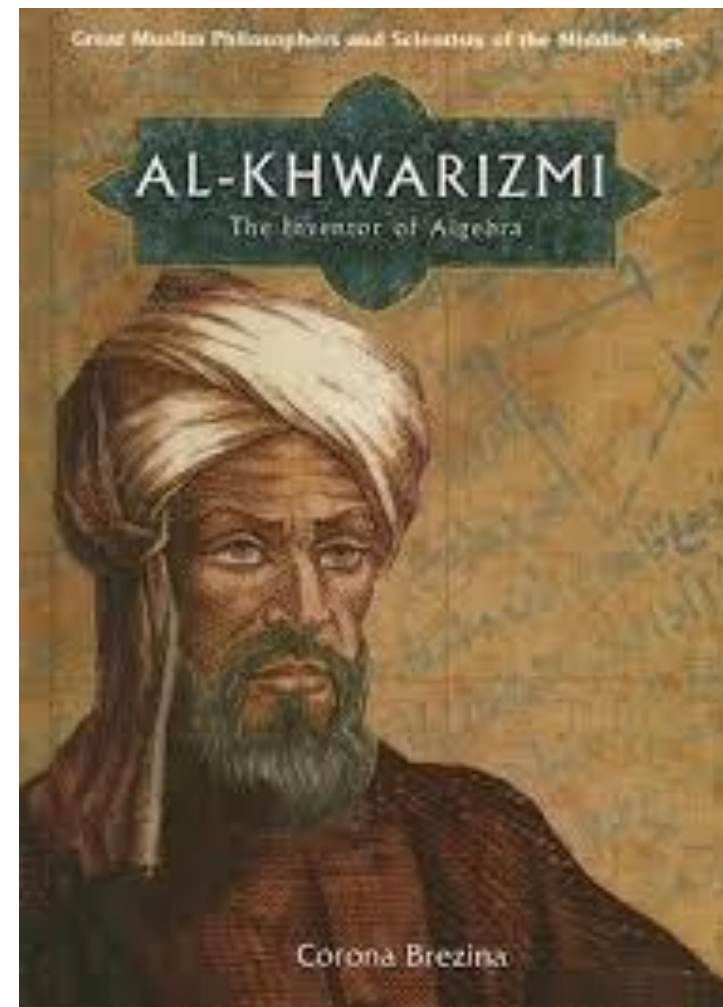
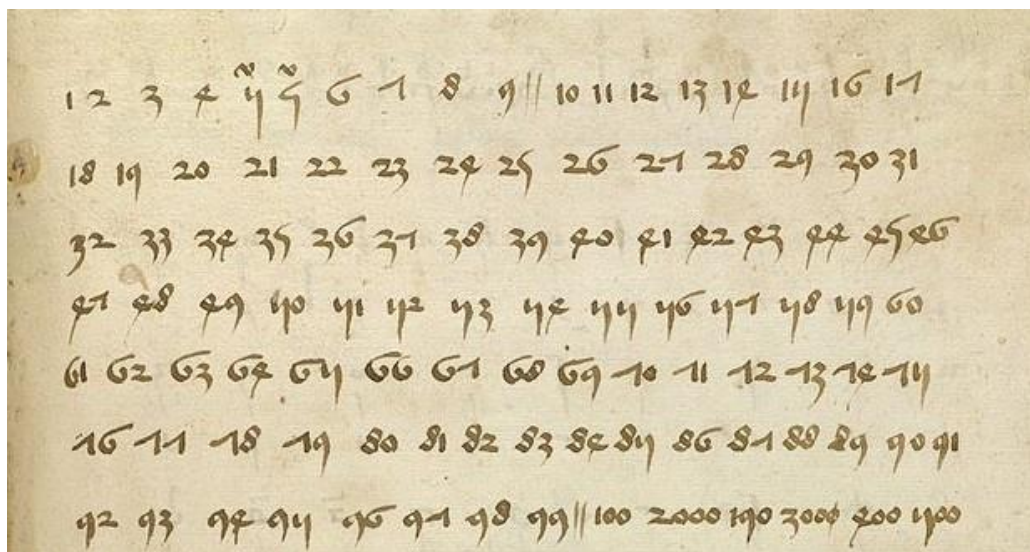
Algoritmus

Al-Khwārizmī: Jak počítat s indickými číslicemi

latinský překlad:



Algoritmi de numero Indorum



Formální definice

Výpočetní problém V

- Obecný způsob/model transformace vstupních dat na výstupní s požadovanými vlastnostmi

Algoritmus A

- Obecný výpočetní postup pro řešení výpočetního problému V

Instance problému I

- Přesně specifikovaný výpočetní problém V s konkrétními daty

Příklad – řazení čísel

- Výpočetní problém V :
 - Vstup: $(\alpha_1, \alpha_2, \dots, \alpha_n)$
 - Výstup: $(a_1, a_2, \dots, a_n)$,
 - Vlastnosti: $a_1 \leq a_2 \leq \dots \leq a_n$
- Algoritmus A : QuickSort
- Instance I problému V : vstupem jsou přirozená čísla (105, 88, ..., 4)

Algoritmus, obecné vlastnosti

Hromadnost

- Je schopen zpracovat libovolně velká vstupní data

Determinovanost

- Každý krok je jednoznačně definován

Konečnost

- Výpočet je vždy řádně ukončen

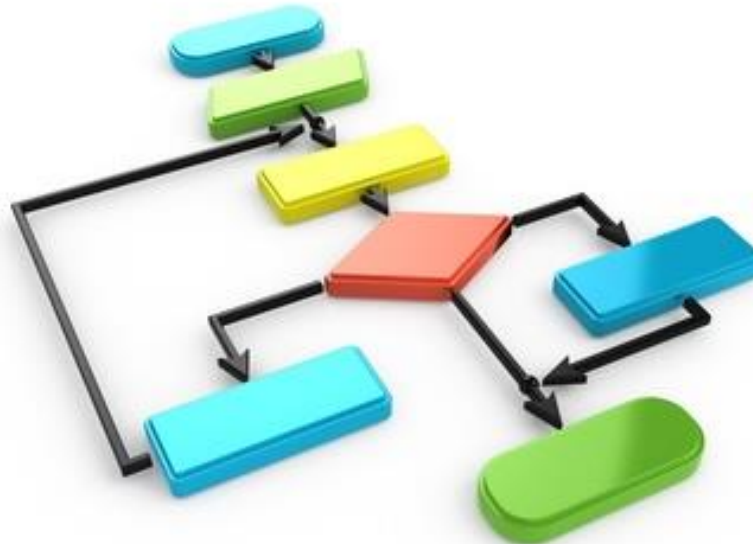
Rezultativnost

- Výpočet vede k požadovaným výsledkům

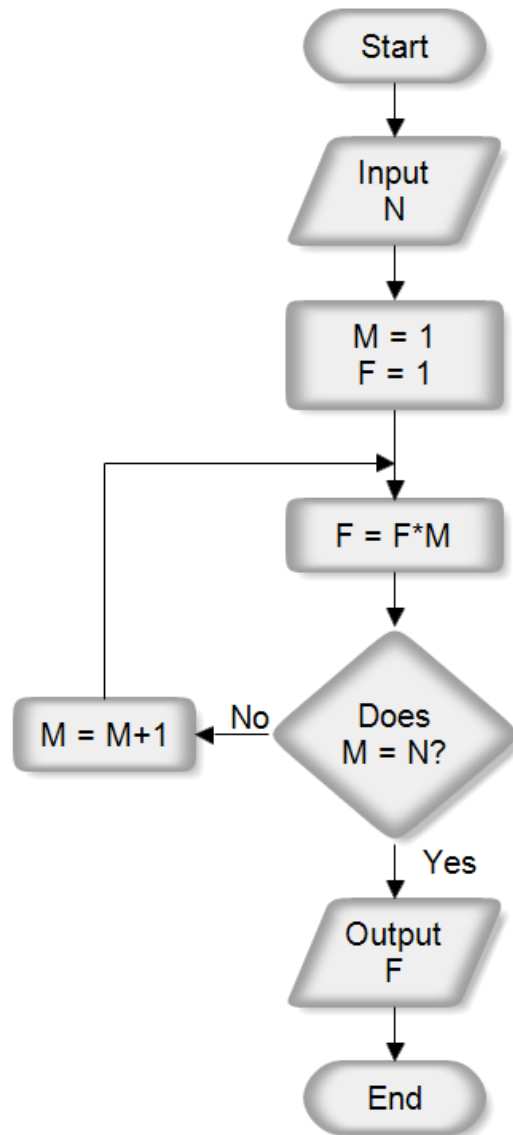
V praxi usilujeme o návrh **časově** nebo **paměťově OPTIMÁLNÍCH** algoritmů

Algoritmy, reprezentace

- Vývojové diagramy (flowchart, control flow diagram)
 - Používané i v SWE nebo procesním řízení
- Implementačně nezávislý pseudokód
 - Běžný nástroj strukturované komunikace



Vývojový diagram a pseudokód



GET value of N

SET initial value of F to 1

SET initial value of M to 1

WHILE M < N

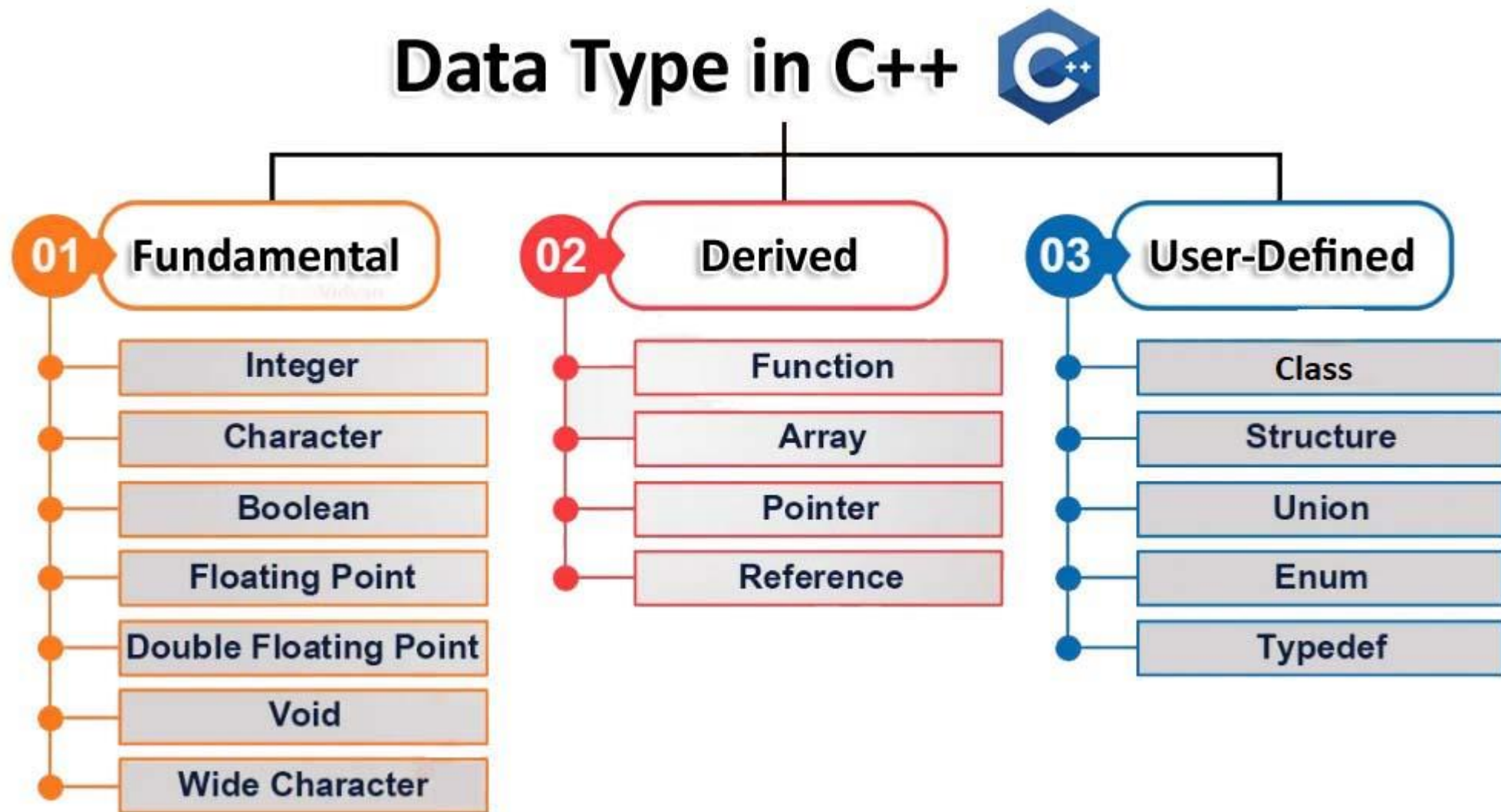
$F = F * M$

$M = M + 1$

ENDWHILE

DISPLAY value of F

Základní datové typy a struktury jazyka C++



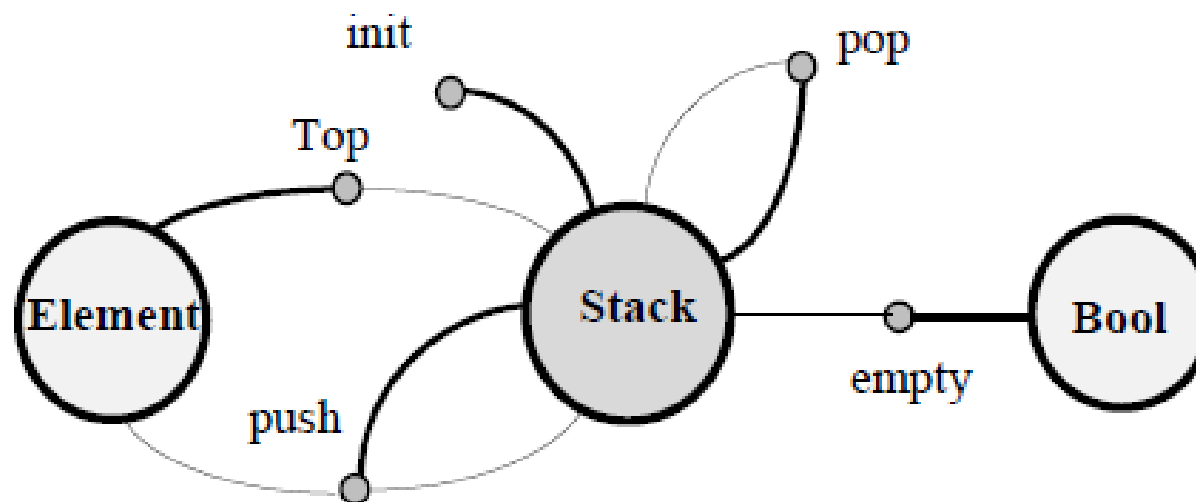
Zdroj: <https://www.sitesbay.com/cpp/cpp-datatype>

Abstraktní datové typy (ADT) a základní operace s nimi

- Vytvoření a zrušení ADT
- Zjišťování vlastností ADT
- Způsoby procházení ADT
- Přidání a odebrání prvku do ADT
 - Na začátek
 - Na konec
 - Na libovolnou pozici
- Vyhledávání a modifikace prvku v ADT (funkce, přetížené operátory)
- Aplikace uživatelských nebo knihovních algoritmů na prvky ADT

ADT zásobník, příklad možné formální reprezentace

- Grafická



- Textová

`init: → stack`

`push(_, _): prvek, stack → stack`

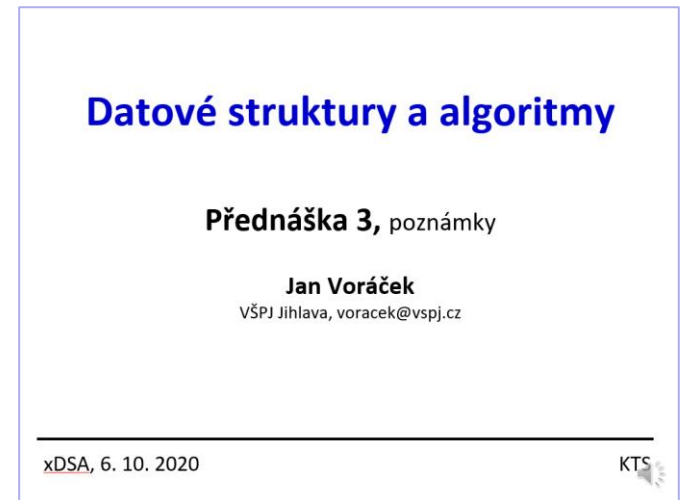
`pop(_): stack → stack`

`empty(_): stack → bool`

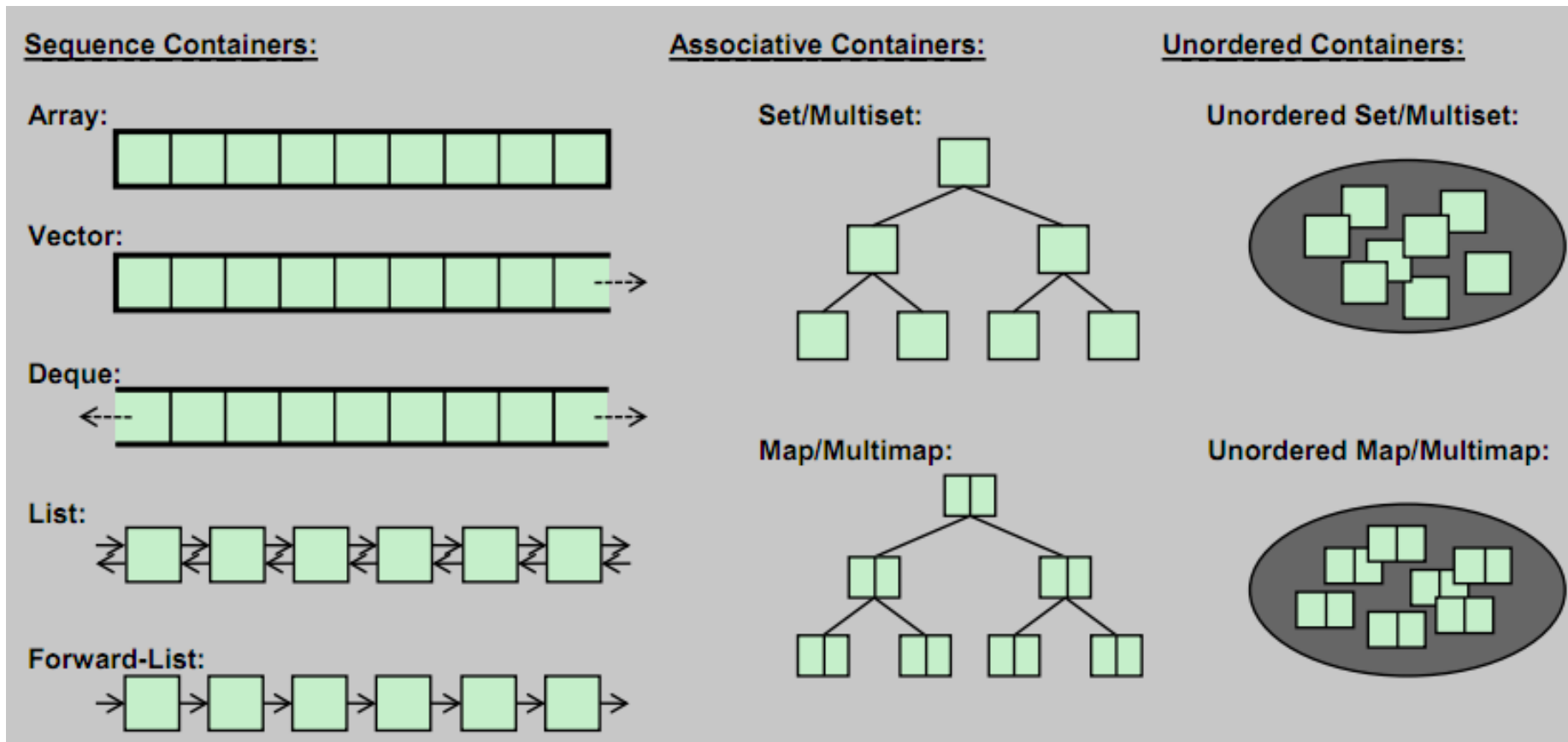
`top(_): stack → prvek`

Dělení ADT (zde začínají AV přednášky 2020 v MS Teams - P3)

- Podle implementační datové struktury
 - Lineární (např. pole, seznamy)
 - Nelineární (např. stromy, *grafy*)
- Podle způsobu přístupu k jednotlivým prvkům
 - Sekvenční (přístup prostřednictvím ostatních prvků)
 - Asociativní (přístup prostřednictvím klíče nebo indexu)
- Podle uspořádanosti prvků
 - Neuspořádané (množina)
 - Uspořádané (seznam, mapa)



ADT, klasifikace 1



SET: unikátní prvky, prvek nemůže být modifikován (const), pouze vložen nebo odstraněn

MULTISET: duplicitní prvky

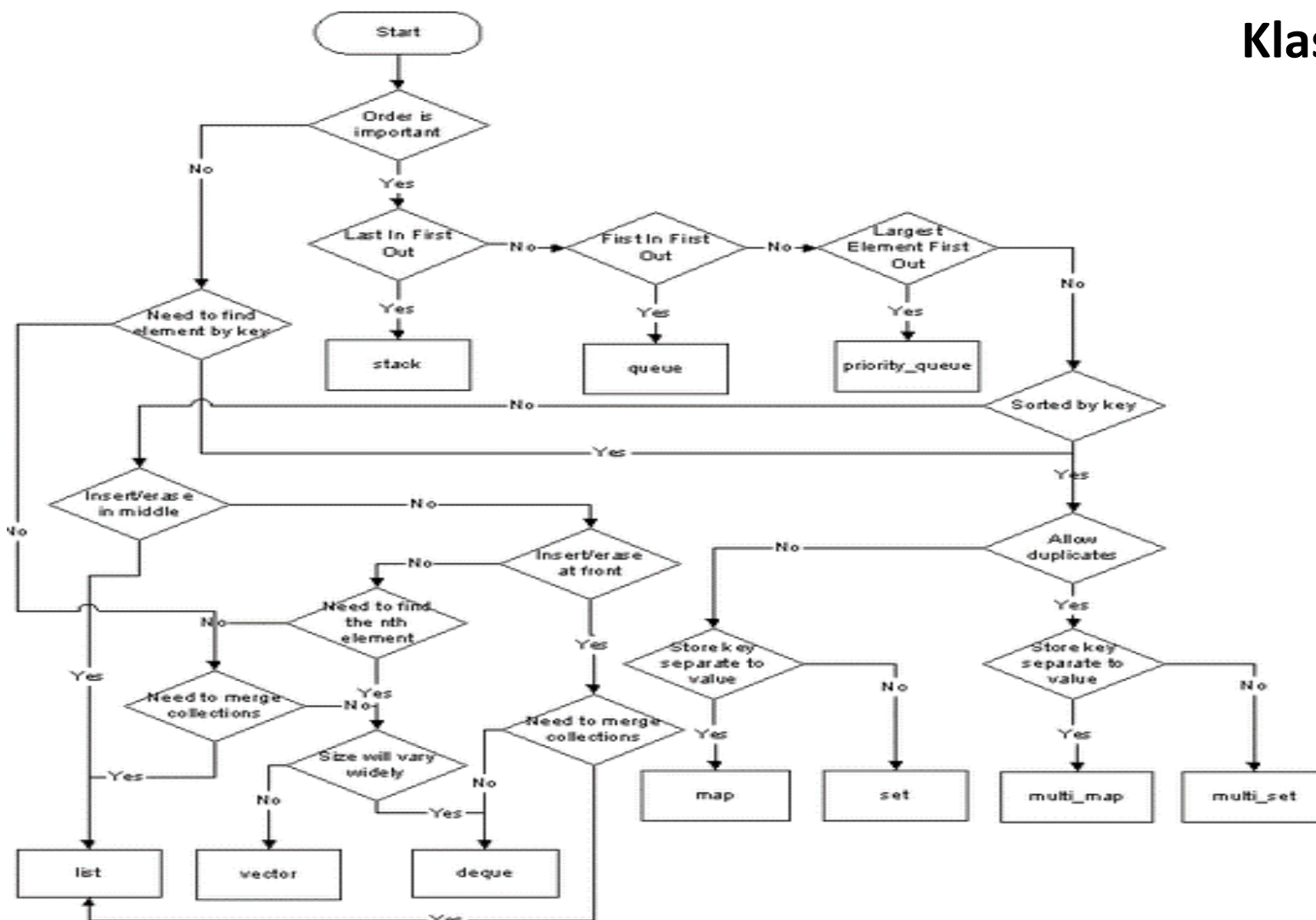
MAP: unikátní klíče, duplicitní prvky

MULTIMAP: prvky nemusí mít unikátní klíče, tj. klíč může mít asociováno více prvků

Zdroj: <http://www.cnblogs.com/rogerroddick/archive/2013/05/10/3071616.html>

ADT, klasifikace 2

Klasifikační priority:
Seřazení
Nutnost klíče
Způsob přístupu
Duplicitní prvky



Zdroj: <http://stackoverflow.com/questions/471432/in-which-scenario-do-i-use-a-particular-stl-container>

Kontejnery v STL knihovně jazyka C++ a jejich implementace

- **Sekvenční kontejnery**

- array (statické pole)
- vector (dynamické pole)
- deque (dynamické pole SS)
- forward_list (SS)
- list (SS)

- **Kontejnerové adaptéry**

- stack (deque nebo vector)
- queue (deque nebo list)
- priority_queue (binární halda)

- **Asociativní kontejnery**

- set
- multiset
- map
- multimap

(vyvážený BVS)

- **Neseřazené asociativní kontejnery**

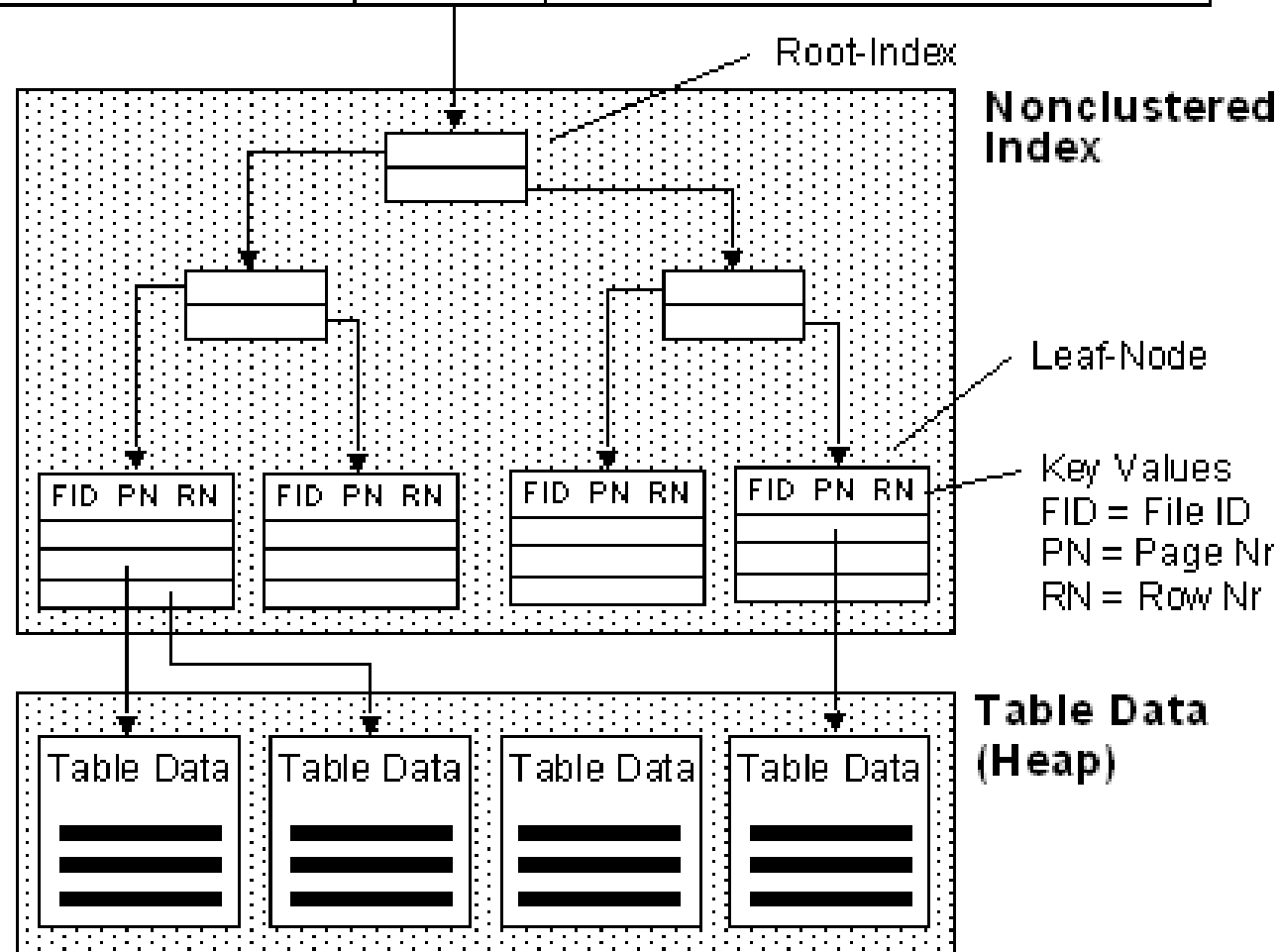
- unordered_set
- unordered_multiset
- unordered_map
- unordered_multimap

(hašovací tabulka)

ADT v databázovém/souborovém systému

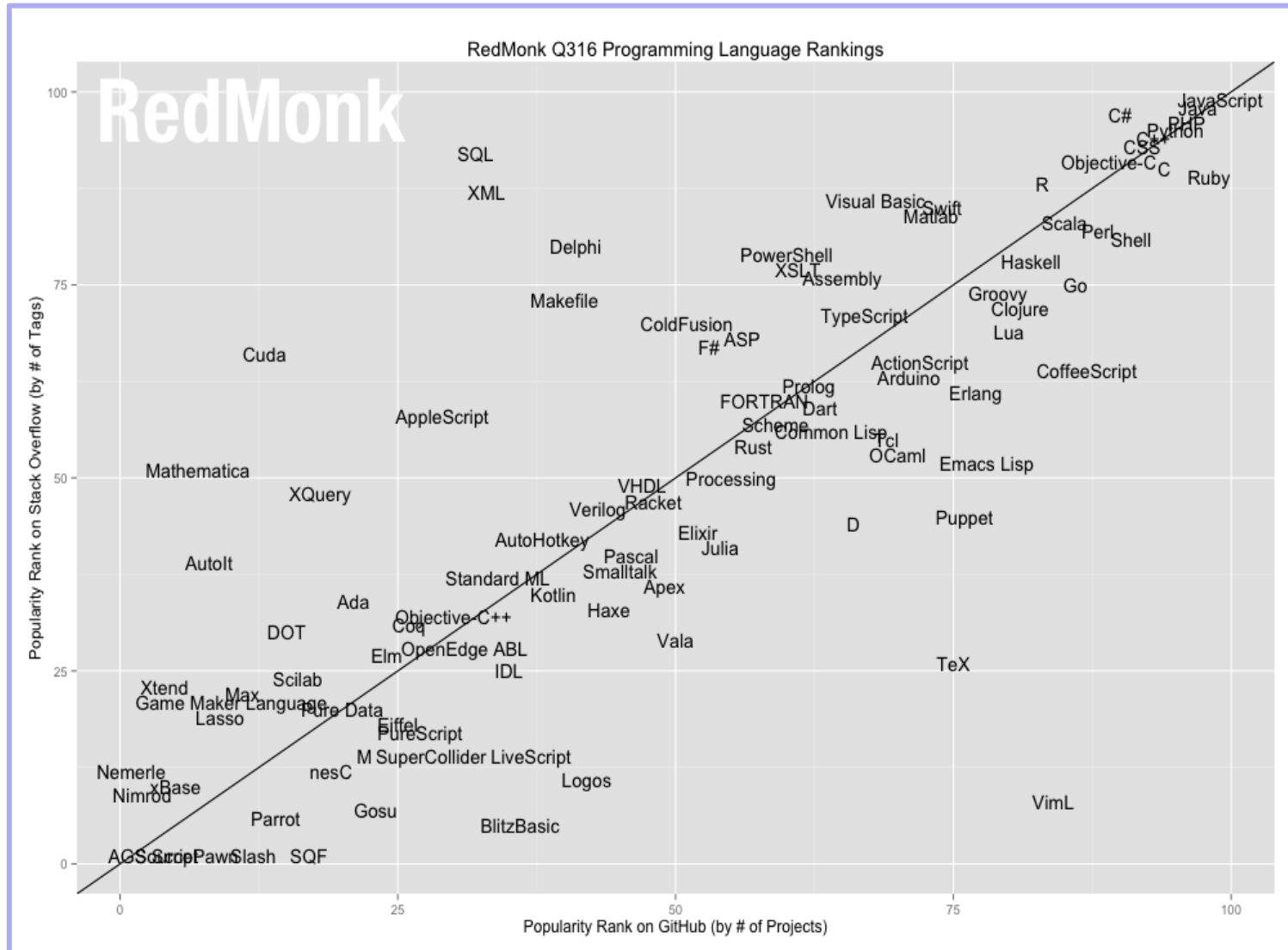
sysindexes

id	indid = 2 to 250	root	
----	------------------	------	--



Zdroj: https://www.akadia.com/services/sqlsrv_data_structure.html

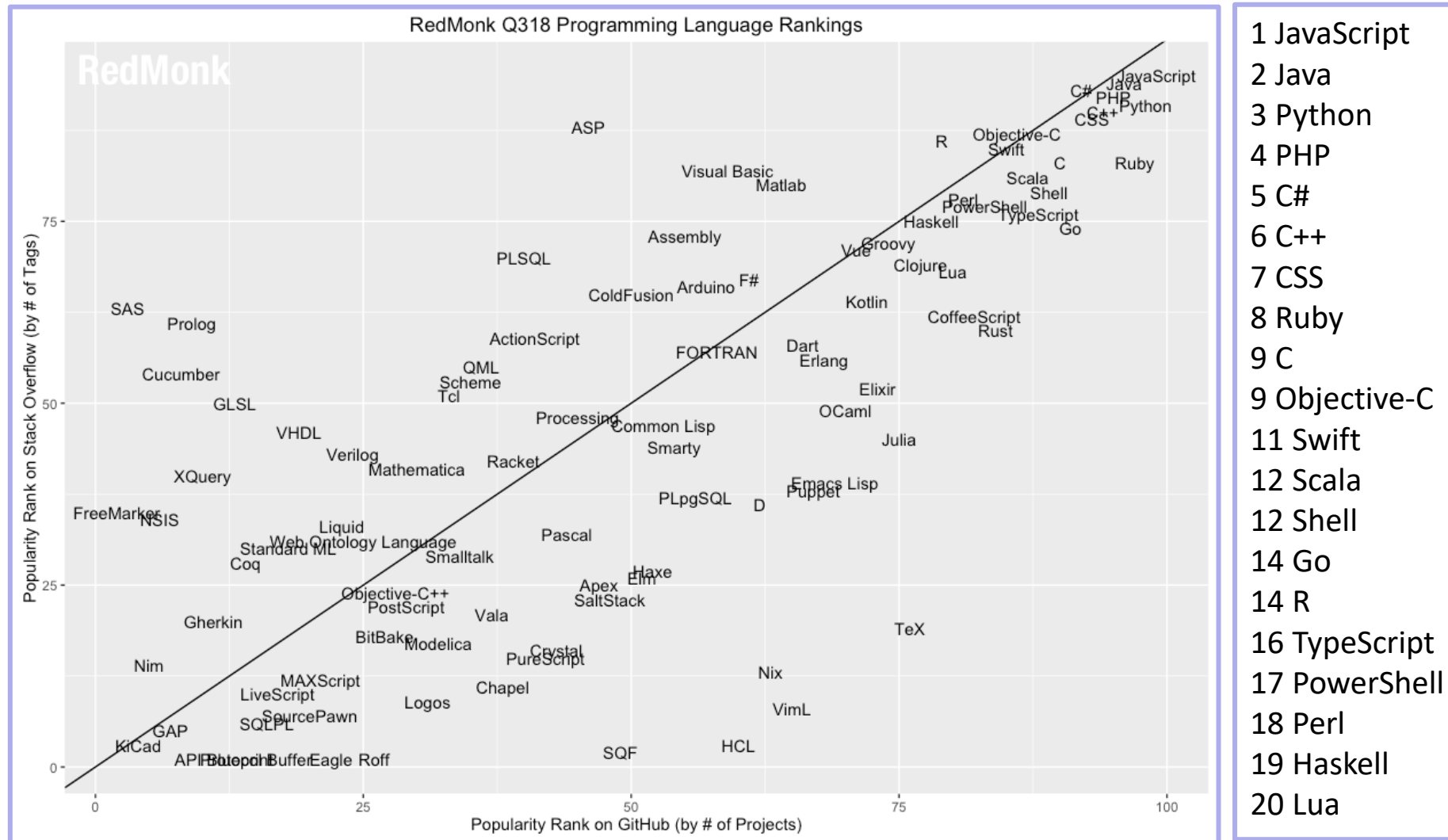
Programovací jazyky, popularita 6/2016



- 1 JavaScript
- 2 Java
- 3 PHP
- 4 Python
- 5 C#
- 5 C++
- 5 Ruby
- 8 CSS
- 9 C
- 10 Objective-C
- 11 Shell
- 12 R
- 13 Perl
- 14 Scala
- 15 Go
- 16 Haskell
- 17 Swift
- 18 Matlab
- 19 Visual Basic
- 20 Clojure
- 20 Groovy

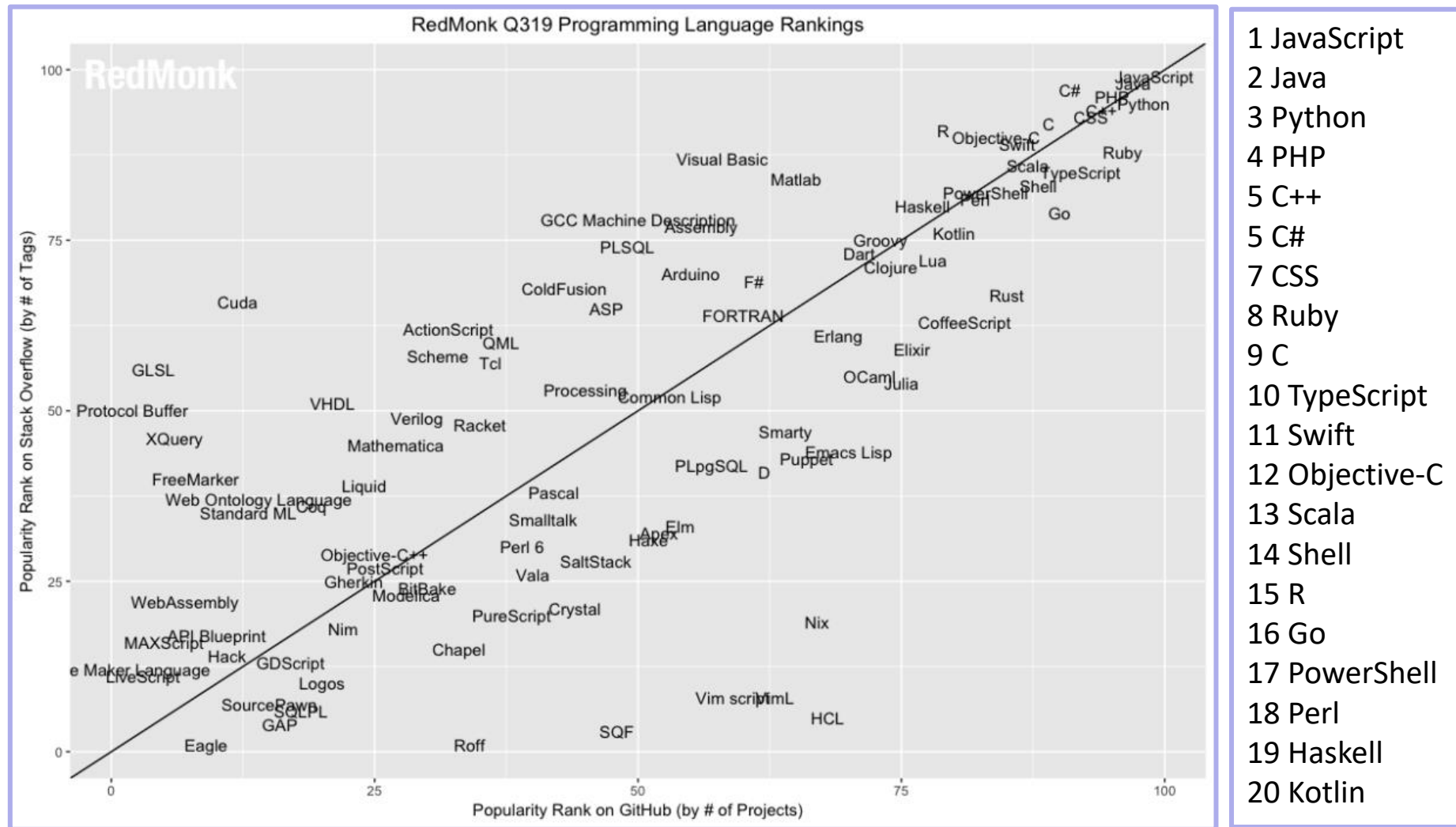
Zdroj: <http://redmonk.com/sograzy/2016/07/20/language-rankings-6-16/>

Programovací jazyky, popularita 6/2018



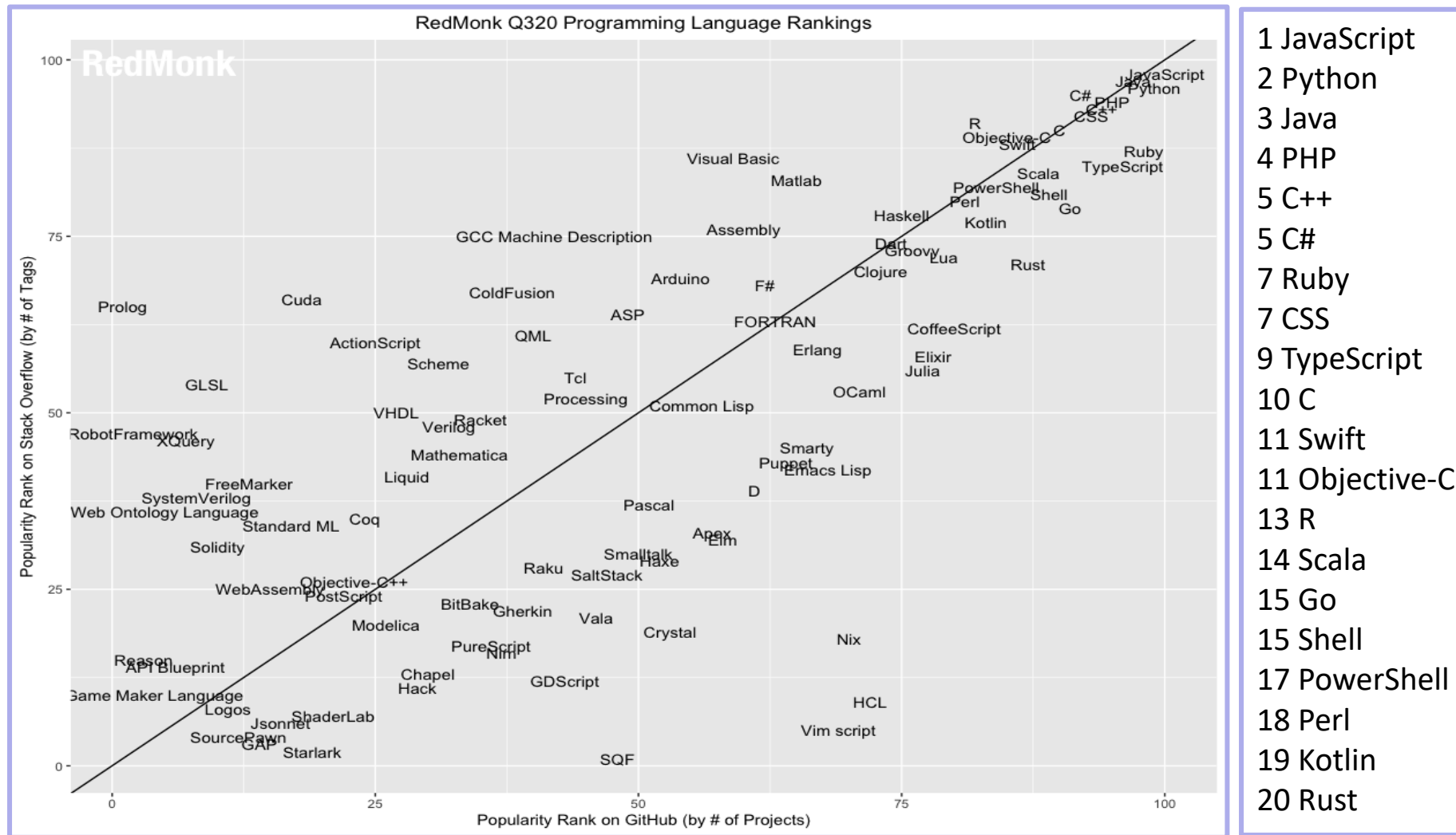
Zdroj: <https://redmonk.com/sogrady/2018/08/10/language-rankings-6-18/>

Programovací jazyky, popularita 6/2019



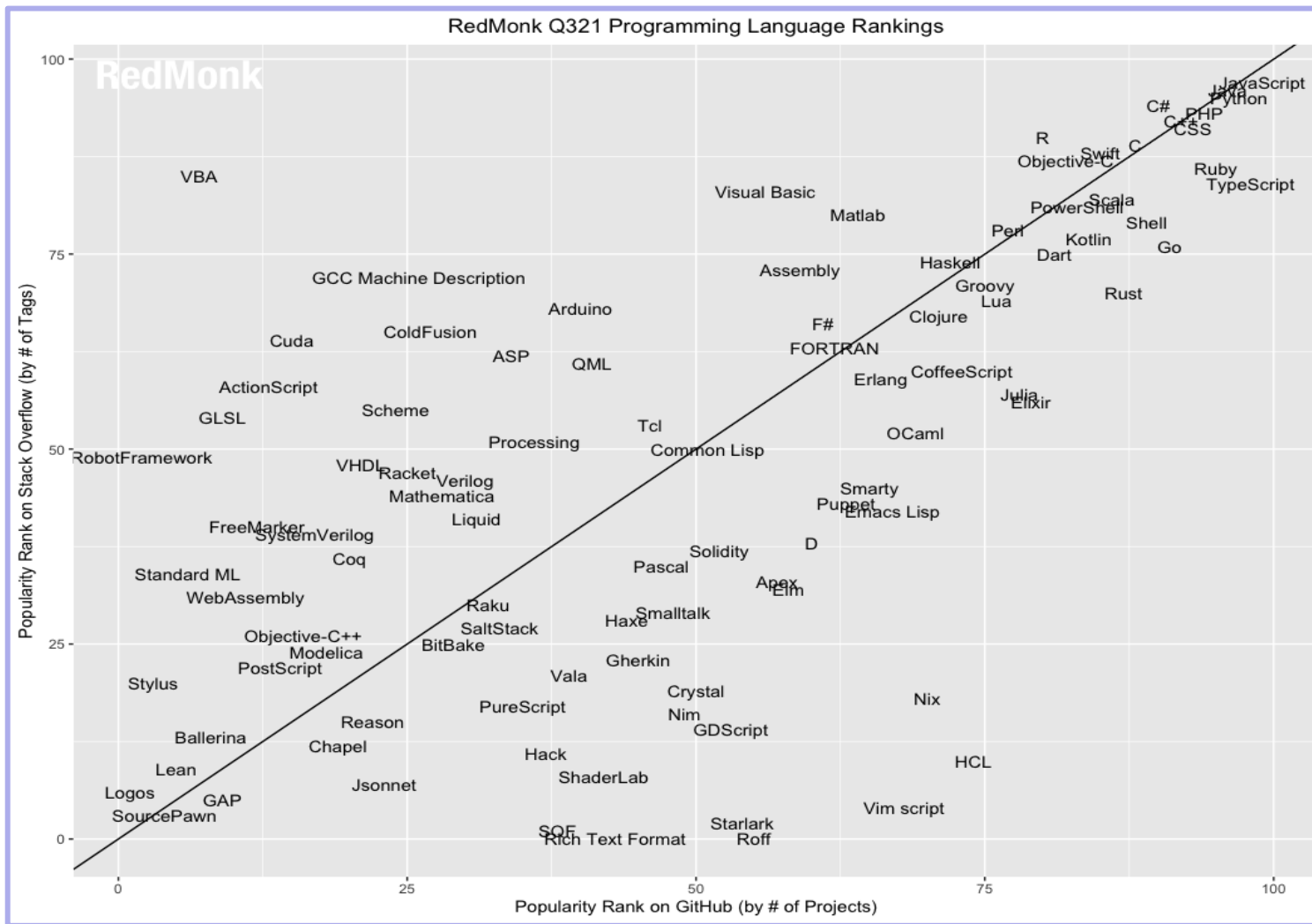
Zdroj: <https://redmonk.com/sogrady/2019/07/18/language-rankings-6-19/>

Programovací jazyky, popularita 6/2020



Zdroj: <https://redmonk.com/sogrady/2020/07/27/language-rankings-6-20/>

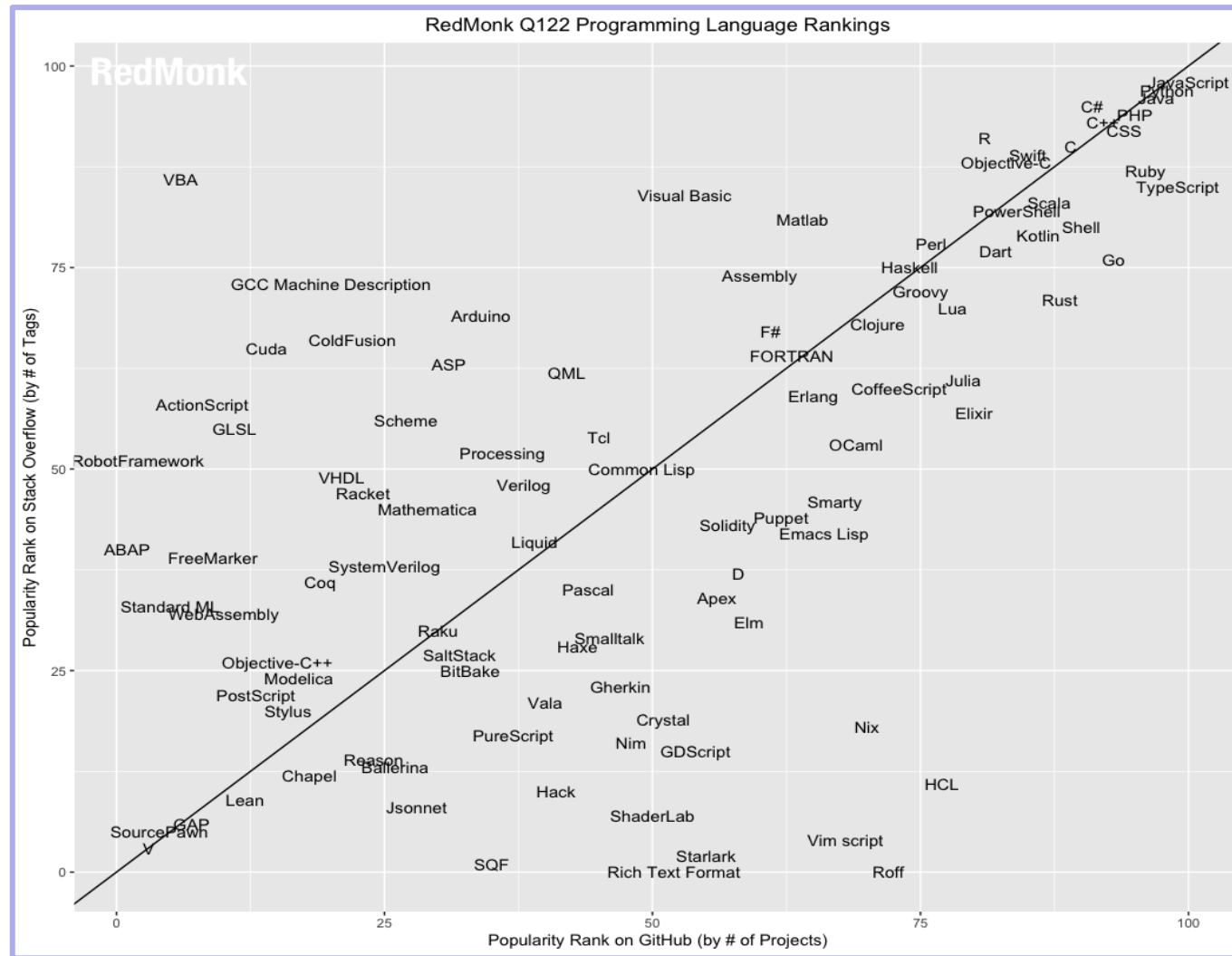
Programovací jazyky, popularita 6/2021



- 1 JavaScript
- 2 Python
- 2 Java
- 4 PHP
- 5 CSS
- 5 C++
- 5 C#
- 8 TypeScript
- 9 Ruby
- 10 C
- 11 Swift
- 12 R
- 13 Objective-C
- 14 Shell
- 14 Scala
- 16 Go
- 17 PowerShell
- 18 Kotlin
- 19 Rust
- 20 Dart

Zdroj: <https://redmonk.com/sogrady/2021/08/05/language-rankings-6-21/>

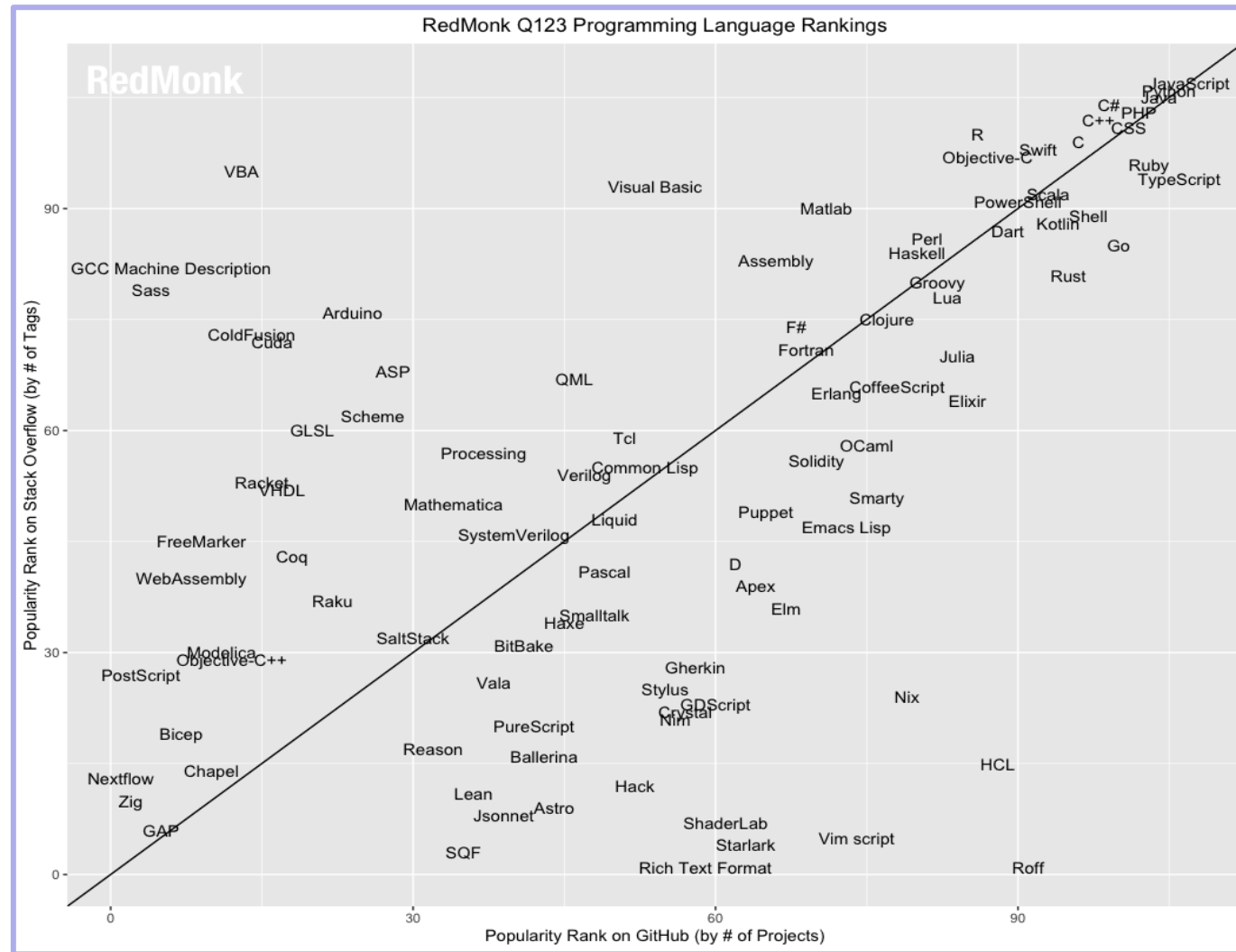
Programovací jazyky, popularita 3/2022



- 1 JavaScript
- 2 Python
- 2 Java
- 4 PHP
- 5 CSS
- 5 C++
- 5 C#
- 8 TypeScript
- 9 Ruby
- 10 C
- 11 Swift
- 12 R
- 13 Objective-C
- 14 Shell
- 14 Scala
- 16 Go
- 17 PowerShell
- 18 Kotlin
- 19 Rust
- 20 Dart

Zdroj: <https://redmonk.com/sogrady/2022/03/28/language-rankings-1-22/>

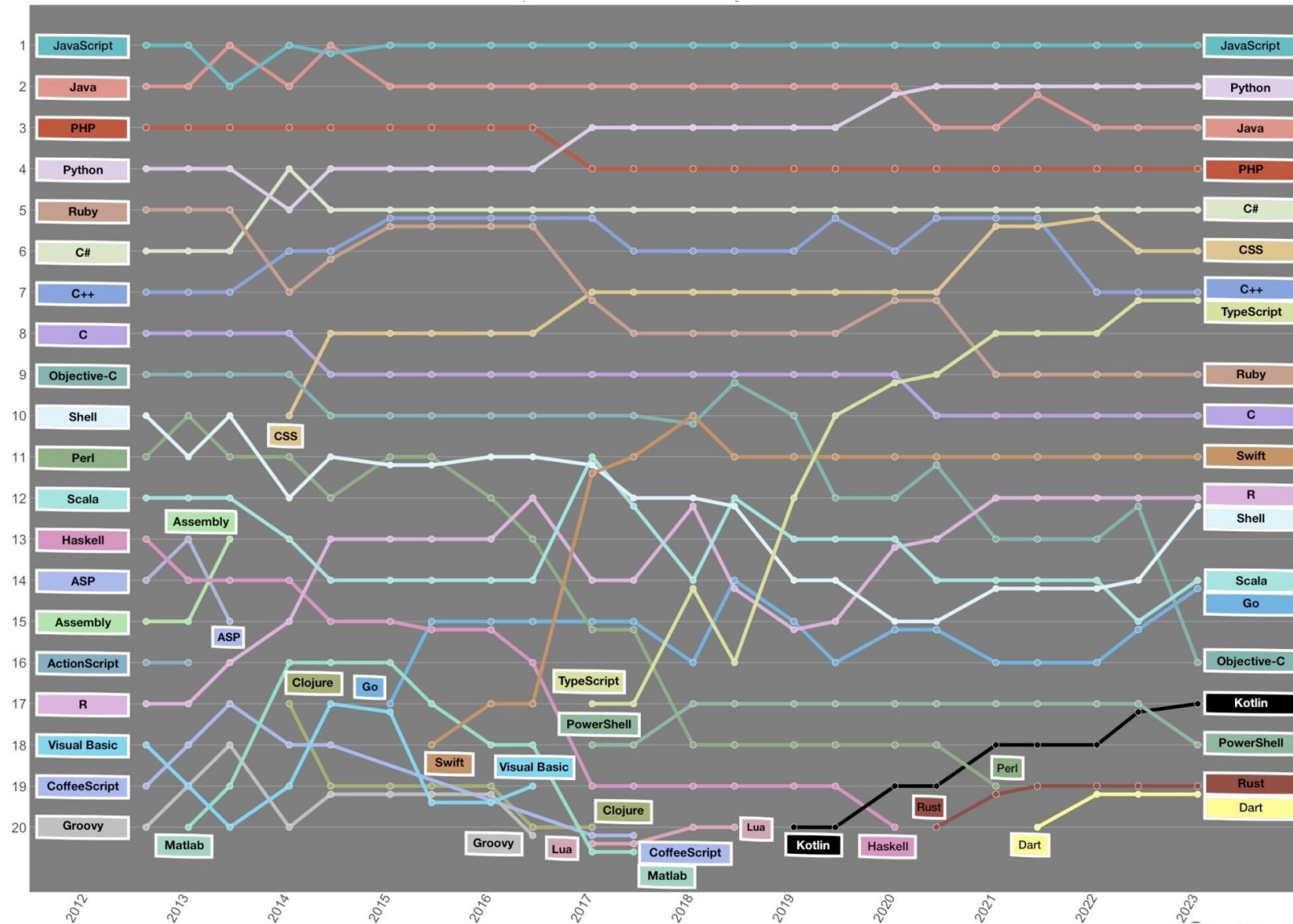
Programovací jazyky, popularita 1/2023



Zdroj: <https://redmonk.com/sogrady/2023/05/16/language-rankings-1-23/>

RedMonk Top 20 Languages Over Time

(Zdroj: <https://tinyurl.com/2kkytjc7>)



Rozdělení programovacích jazyků

- **IMPERATIVNÍ: řeší způsob řízení výpočtu**
 - Řeší především **JAK**
 - Chápou **algoritmy** jako detailní výpočetní předpisy
 - Naivní a nestrukturované (Assembler, Basic, Fortran)
 - Strukturované (Pascal, C, C++, Java)
 - Mají typickou řídicí strukturu: **deklarace** + **příkazy** (větvení, cykly) + **výrazy** (přiřazení), aplikované na **atomické objekty** jazyka
- **DEKLARATIVNÍ (funkcionální): řeší formální logiku výpočtu**
 - Řeší především **CO**
 - Ekvivalentem algoritmů je **Lambda kalkul**
 - Omezená možnost řídicích příkazů a vlastních funkcí
 - Databázové dotazovací jazyky (SQL), zpracování regulárních výrazů, webových stránek (HTML), logická programování (Prolog), funkcionální programování (Haskell, Lisp), jazyky konfiguračního managementu

Řada jazyků má prvky obou přístupů (JavaScript, C#, Python)

Jazyky a datové struktury

Každý programovací jazyk umožňuje práci s datovými strukturami:

- Deklarativní (SQL, Prolog, Lisp)
 - Implicitně
- Aplikačně orientované imperativní jazyky
 - Explicitně, typicky na úrovni knihoven:
 - Skriptovací (php frameworky, Python)
 - Programovací (C#, Java)
- Systémově orientované imperativní jazyky (C, C++)
 - Explicitně na implementační úrovni

1. ADT je reprezentován:
 - a) Třídami a objekty
 - b) Programy a zdrojovými texty
 - c) Třídami a funkcemi
 - d) Operacemi a objekty
 - e) Datovými strukturami a operacemi s nimi, definovanými v rozhraní ADT

2. Po potřeby práce s ADT je třeba znát konkrétní způsob jejich implementace?
 - a) Ano
 - b) Ne

3. Konkrétní způsob implementace ADT se nijak neprojeví na běhových vlastnostech programu.
 - a) Ano
 - b) Ne

-
4. Co to je datová struktura
 - a) Totéž co ADT
 - b) Základní datový typ, definovaný syntaxí programovacího jazyka
 - c) Konkrétní způsob organizace dat v paměti počítače

 5. V jakém z následujících ADT jsou prvky uspořádány podle pořadí jejich příchodu/uložení s tím, duplicitní položky/prvky jsou povoleny?
 - a) Množina
 - b) Multimnožina (Bag)
 - c) Seznam
 - d) Prioritní fronta

 6. V jakém z následujících ADT nejsou prvky nijak uspořádány a duplicitní položky/prvky nejsou povoleny?
 - a) Množina
 - b) Multimnožina (Bag)
 - c) Seznam
 - d) Prioritní fronta

-
7. V jakém z následujících ADT jsou prvky uspořádány podle jejich priority a duplicitní položky/prvky jsou povoleny?
- a) Množina
 - b) Multimnožina (Bag)
 - c) Seznam
 - d) Prioritní fronta
8. V jakém z následujících ADT nejsou prvky nijak uspořádány a duplicitní položky/prvky jsou povoleny?
- a) Množina
 - b) Multimnožina (Bag)
 - c) Seznam
 - d) Prioritní fronta

9. V jakém ADT byste očekávali následující operace: `put`, `get`, `is_empty`, `is_full`, `has_item`, `delete_item`, `total_items`, `empty`:

- a) Zásobník
- b) Fronta
- c) Pole
- d) Množina

10. Který z následujících ADT nemá lineární strukturu

- a) Vector
- b) Stack
- c) Forward list
- d) Queue
- e) Všechny uvedené ADT mají lineární strukturu

11. Které z následujících srovnání pole (ARR) a spojového seznamu (LL) není pravdivé

- a) Velikost pole je konstantní, velikost spojového seznamu je proměnná
- b) Vkládání nových prvků do pole i do spojového seznamu je jednoduché a rychlé.
- c) Ve spojovém seznamu nelze přistoupit k libovolnému prvku s konstantní složitostí
- d) Prvek spojového seznamu je paměťově náročnější než prvek pole.

12. Do prázdného spojového seznamu přidáváme vždy na jeho konec (append) postupně prvky 10, 20 a 30. Jaký bude jeho výsledný obsah:

- a) 20 --> 10 --> 30
- b) 20 --> 30 --> 10
- c) 10 --> 20 --> 30
- d) 30 --> 20 --> 10

13. Ze spojového seznamu 10 --> 20 --> 30 odstraníme prvek 20. Jaká bude jeho výsledná struktura:

- a) 10 --> 20 --> 30
- b) 20 --> 30
- c) 10 --> 30
- d) 10 --> 20

Přednášky doc. Hudce, část 1 (úvod)

Algoritmy v C++

- Cíl předmětu
 - zdokonalit se v návrhu a implementaci abstraktních datových typů
 - seznámit se s efektivními algoritmy operací nad ADT a naučit se sestavovat algoritmy řešení složitějších problémů
 - naučit se využívat knihovnu STL (Standard Template Library)
- Organizace předmětu
 - 2 hod. přednášek
 - 2 hod. cvičení v počítačové učebně
 - 1 test na cvičení
 - domácí úlohy
 - semestrální práce
 - zkouška (2 příklady na papíře + doplňující otázka)
- Studijní materiály a další informace budou postupně zveřejňovány na
S:UCITELE/Hudec/AVC

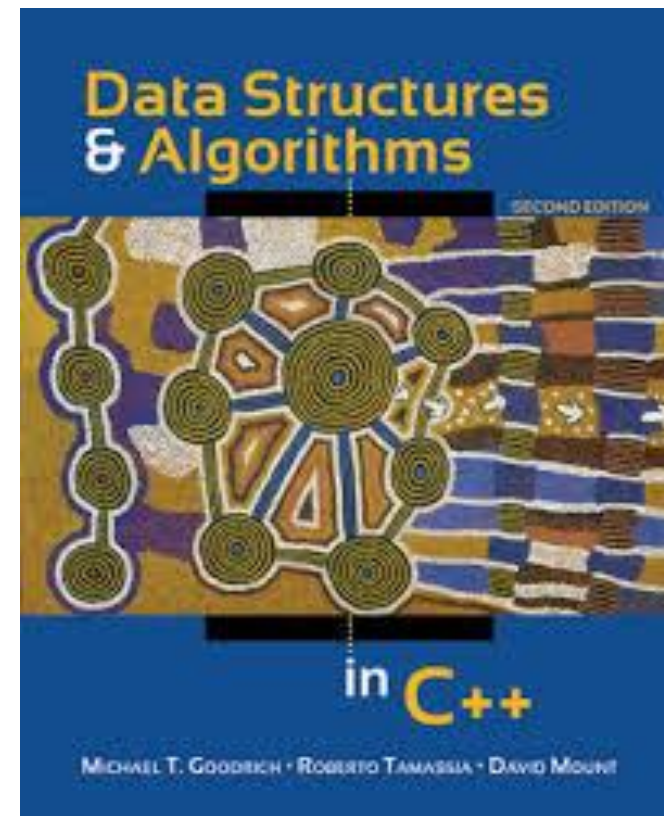
Materiály Goodrich, Tamassia, Mount (**GTM**)

- Jedná se o doprovodné slajdy k učebnici:

Goodrich, Tamassia, Mount:

Data Structures and Algorithms in C++

2nd Edition, Wiley, 2011



dostupné na <https://bit.ly/3kBeLXB> v souboru odkazovaném jako *PDF handouts*.